Change Stack

Do / Say / See

Understand the feature through layers, findings, a code-level question and a review decision.

5 pillars | 20 core pages | limit 25

Use demo-site-pr-examples PR #4. Check Review history before reading findings. The discount thread is historical and resolved; keep Resolved visible to find it.

Read each grouped step top to bottom. Do contains the clicks and scrolls; read only Say aloud. See is a frame from the delivered video. Times refer to that video. The narration is preserved in full, including the recorded opening and closing.

Creation, Send and approval actions describe the recorded flow. Rehearse with the linked existing results; use a separate training run when repeating those actions. Live pages may have changed. Resource links require the corresponding account access.

OPEN THE DEMO

Flagship PR #4

PR #4: Change Stack

PR #4: SQL finding

PR #4: historical discount thread

PILLARS AND PAGE LIMITS

Introduction - 1 page

Open the Stack - 3 steps / pages

Walk the Layers - 4 steps / pages

Filter Findings - 4 steps / pages

Ask in Context - 4 steps / pages

Approve & Merge - 4 steps / pages

The core count includes the opening page when present. This cover, operator prompts and resource directory are outside the core. Every pillar uses at most four grouped steps.

INTRODUCTION / OPENING / 00:00-00:20

Introduce the flow

DO

Show the section’s flow slide. Keep the full slide visible and highlight each pillar as it comes up in the introduction.

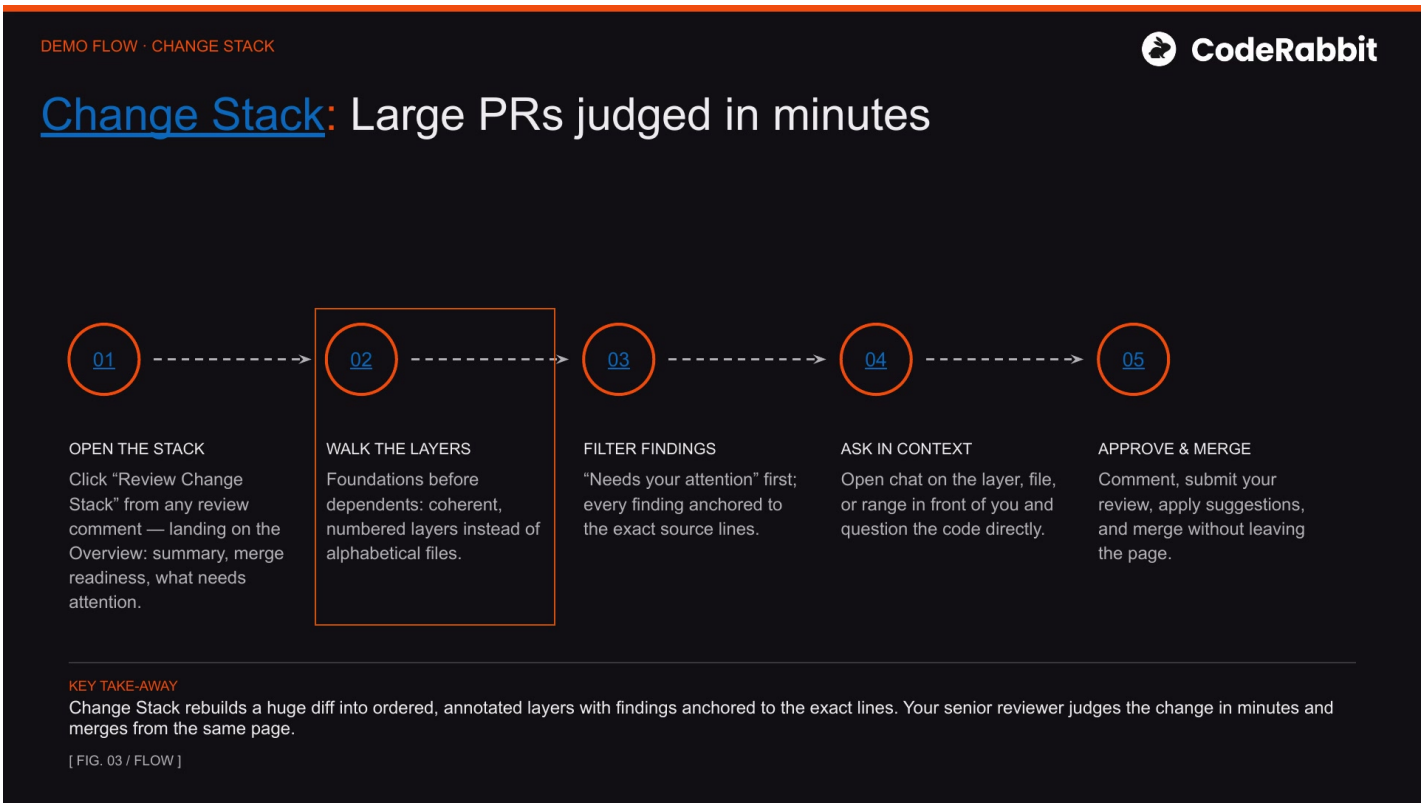
At the end of the introduction, switch to the first demo screen.

Slide deck

SAY

Once I've chosen a pull request, I want to understand how its changes fit together. I'll open Change Stack, walk through the layers and connect the invoice search and discount behavior to the code and tests. Then I'll filter the findings and ask a question about the relevant code. We'll finish with the review options and the conditions that need to be met before a merge.

SEE / VIDEO FRAME AT 00:09



OPEN THE STACK / STEP 1 OF 3 / 00:21-00:29

Enter from Triage

DO

Open PR #4's Triage card and click Review this pull request in Change Stack.

Switch to the new tab and confirm the PR identity.

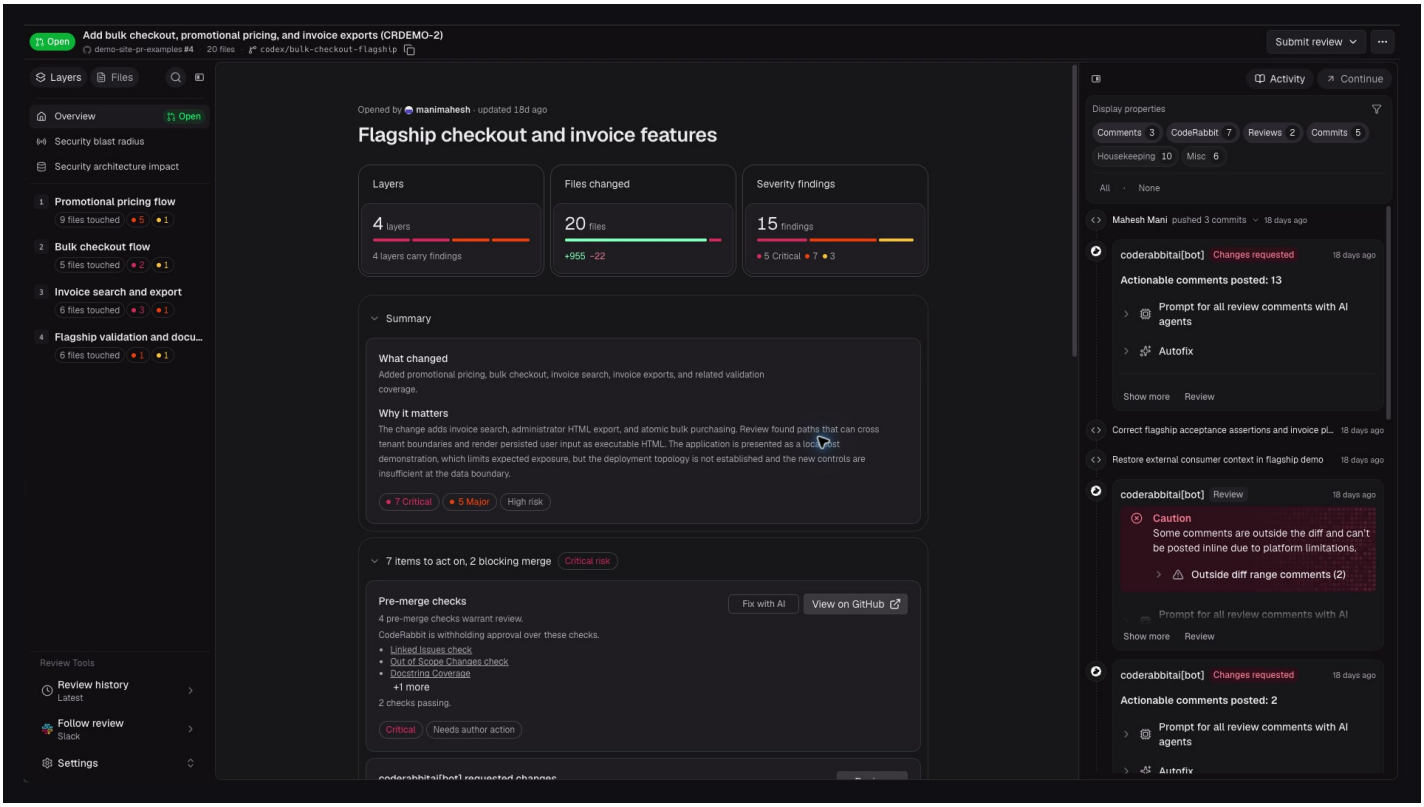
Triage queue

PR #4: Change Stack

SAY

From the pull request card in Triage, I open Change Stack. This is our checkout and invoice change and overview is the starting point.

SEE / VIDEO FRAME AT 00:27



OPEN THE STACK / STEP 2 OF 3 / 00:29-00:42

Read Overview and attention items

DO

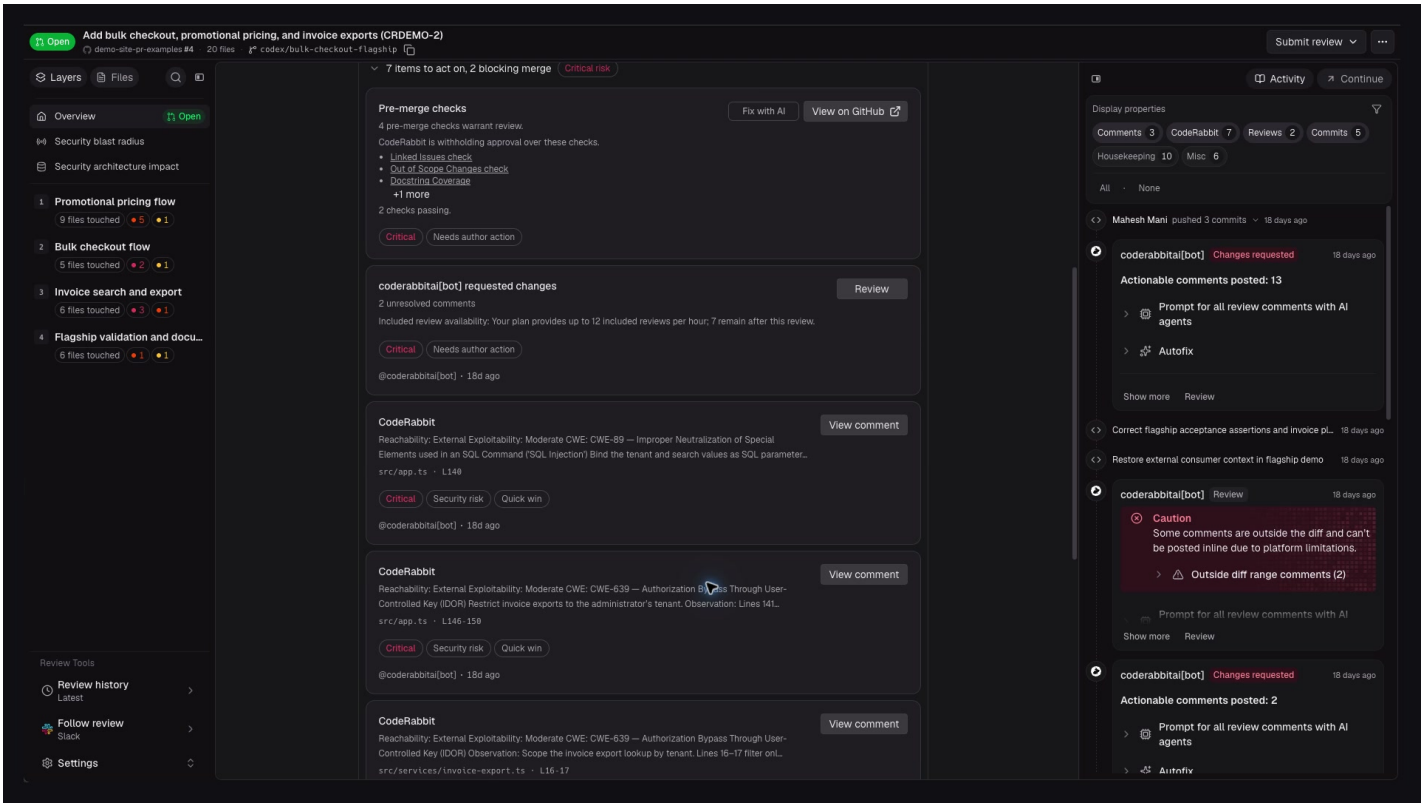
Click Overview in the left rail. Scroll through the summary and changed behavior.

Continue to What changed and remaining concerns; expand details where needed.

SAY

I read the summary to understand what the feature is trying to do and then look at the areas that need attention. For this application, I'm checking two promises. Customers should see their own invoices and qualifying orders should receive a discount.

SEE / VIDEO FRAME AT 00:40



OPEN THE STACK / STEP 3 OF 3 / 00:42-00:54

Check the review snapshot

DO

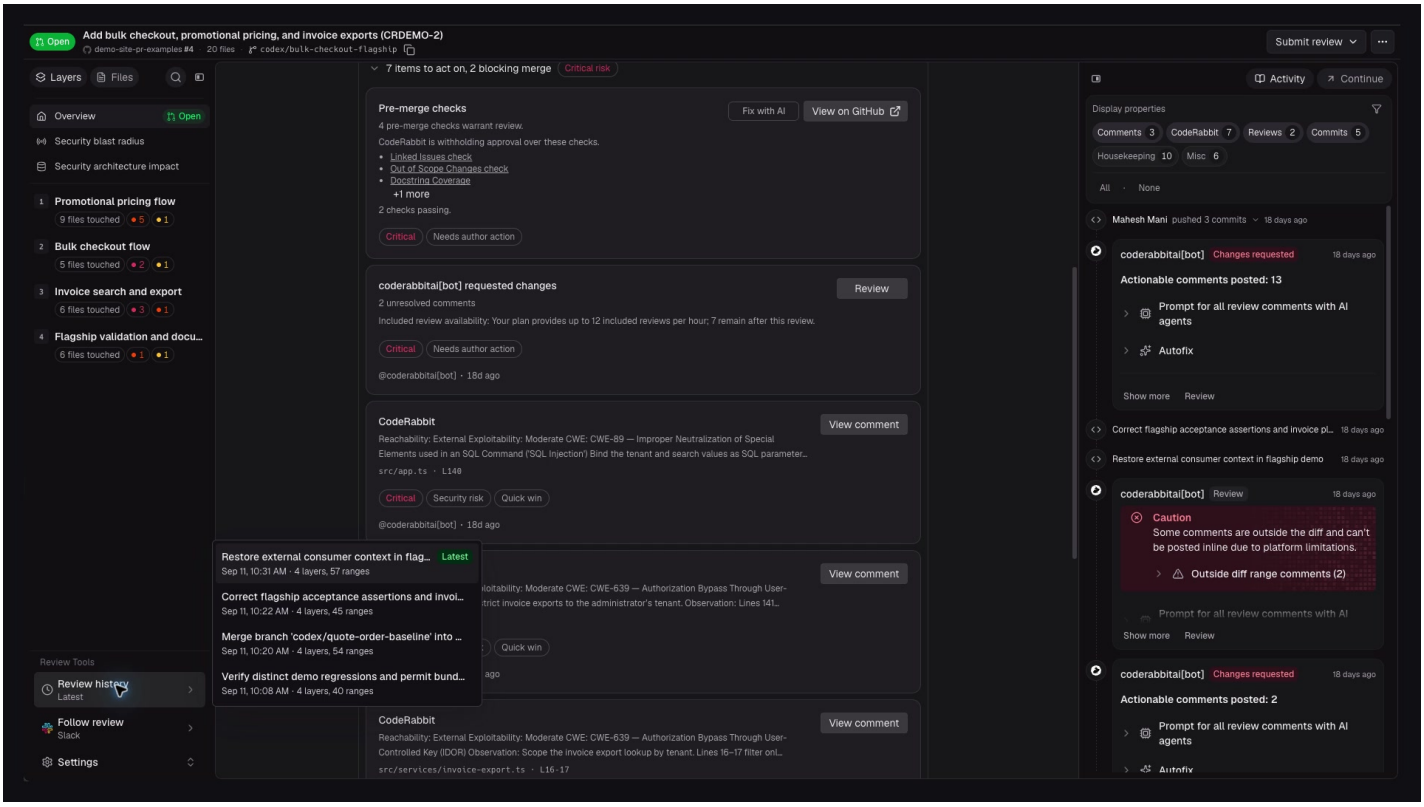
Open Review history at the lower left and inspect the recorded review version.

Select the historical snapshot if needed to find the retained discount discussion. Do not regenerate a review.

SAY

I also check which review version I'm reading. The findings belong to that version of the code. If the author pushes an update, I need to inspect the newer review before deciding whether the issue is addressed.

SEE / VIDEO FRAME AT 00:51



WALK THE LAYERS / STEP 1 OF 4 / 00:55-01:06

Open pricing and checkout

DO

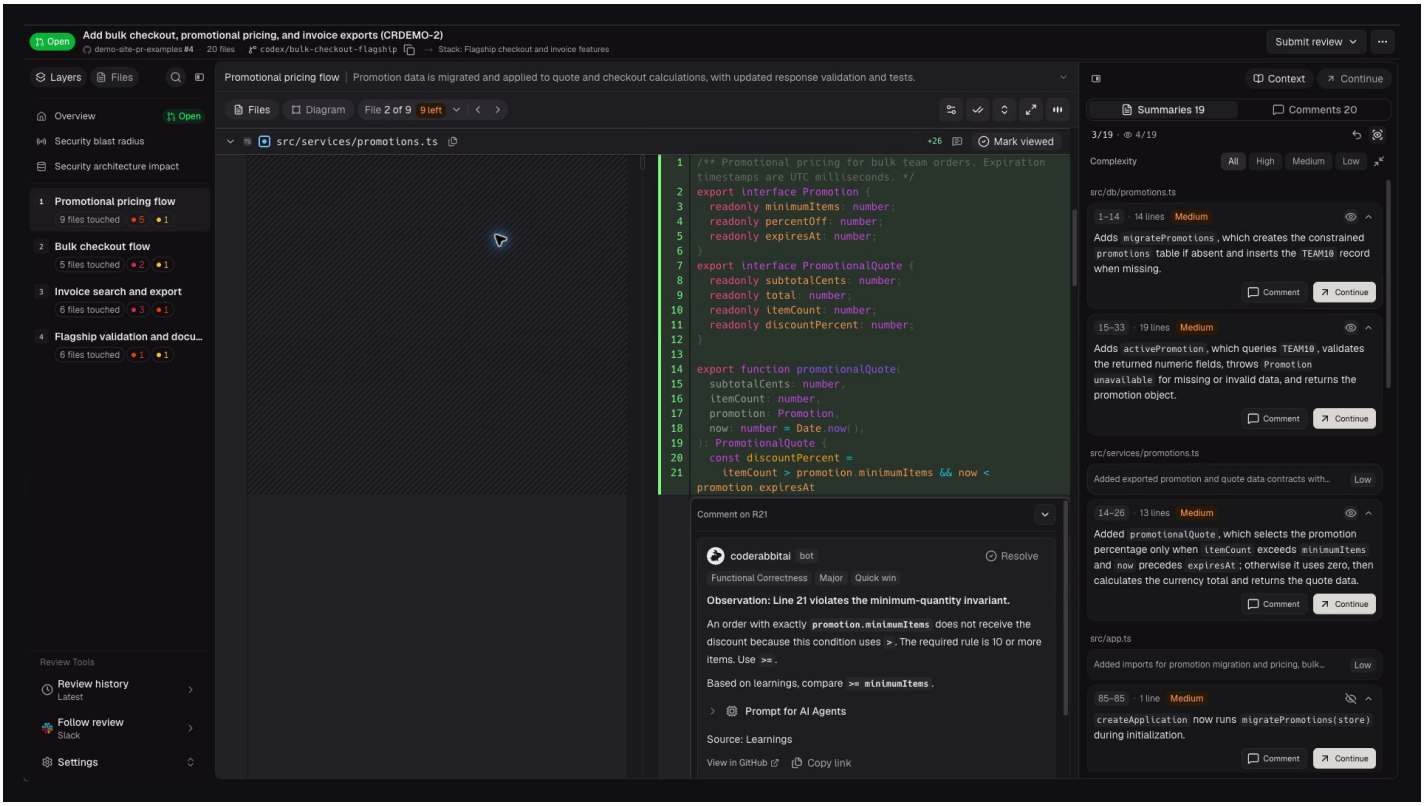
Click the promotional-pricing layer in the left rail. Select src/services/promotions.ts in its file list.

Read the summary and adjacent code. Then click the bulk-checkout layer and inspect its related changes.

SAY

I open the layers to work through related changes in smaller groups. The promotion pricing layer shows how discount eligibility is calculated. Bulk discount shows where that behavior is used.

SEE / VIDEO FRAME AT 01:02



WALK THE LAYERS / STEP 2 OF 4 / 01:07-01:24

Trace invoice search

DO

Click the invoice-search/export layer and choose src/services/invoice-search.ts.

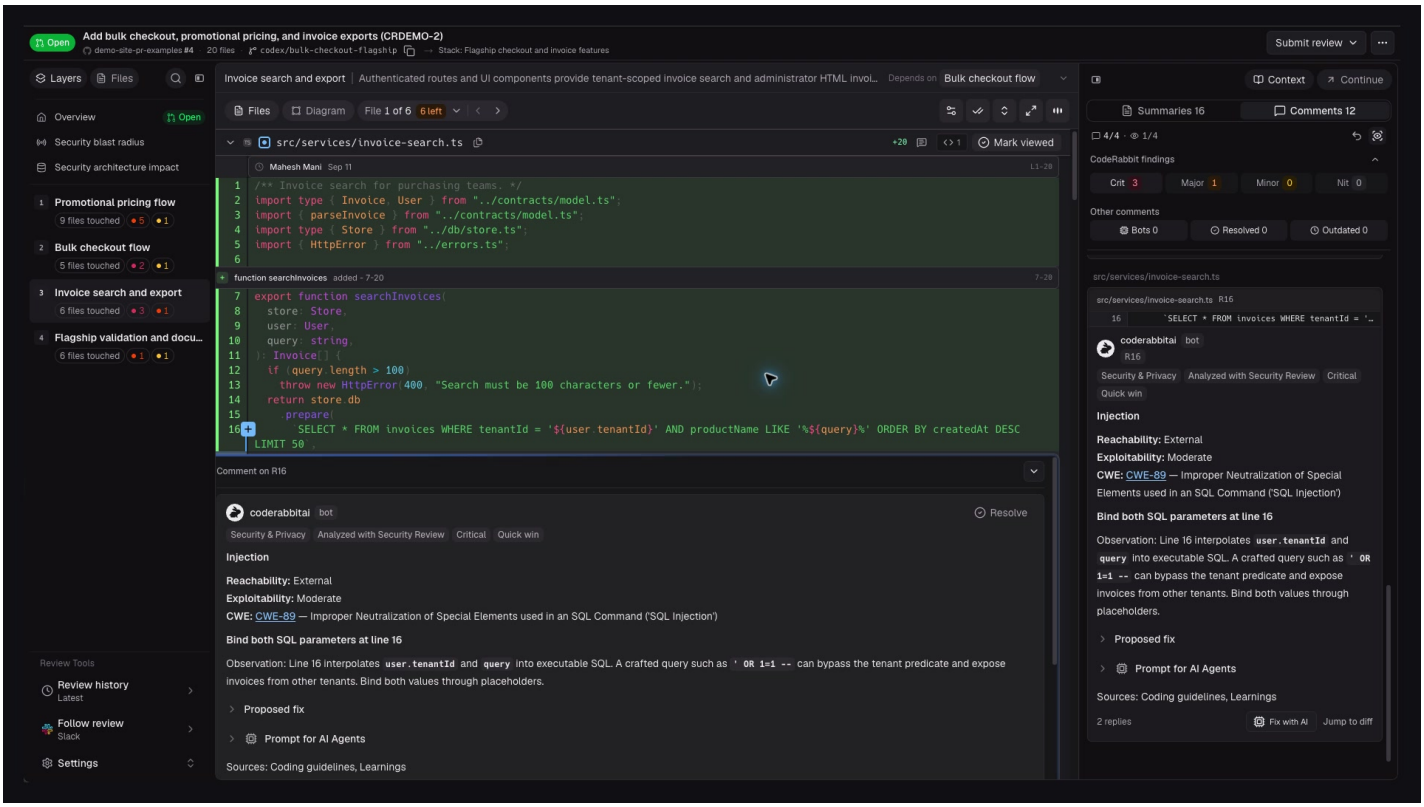
Scroll to the query and the customer/search inputs. Keep the inline finding visible.

PR #4: SQL finding

SAY

Invoice search and export cover another part of the feature with tests and documentation in their own layer. I select the invoice layer and scroll to the search query. The customer supplies search text and the application uses it to retrieve invoices. I check how the text and the customer's identity enter the query.

SEE / VIDEO FRAME AT 01:21



WALK THE LAYERS / STEP 3 OF 4 / 01:25-01:42

Read the historical discount discussion

DO

Return to the pricing layer and promotions.ts.

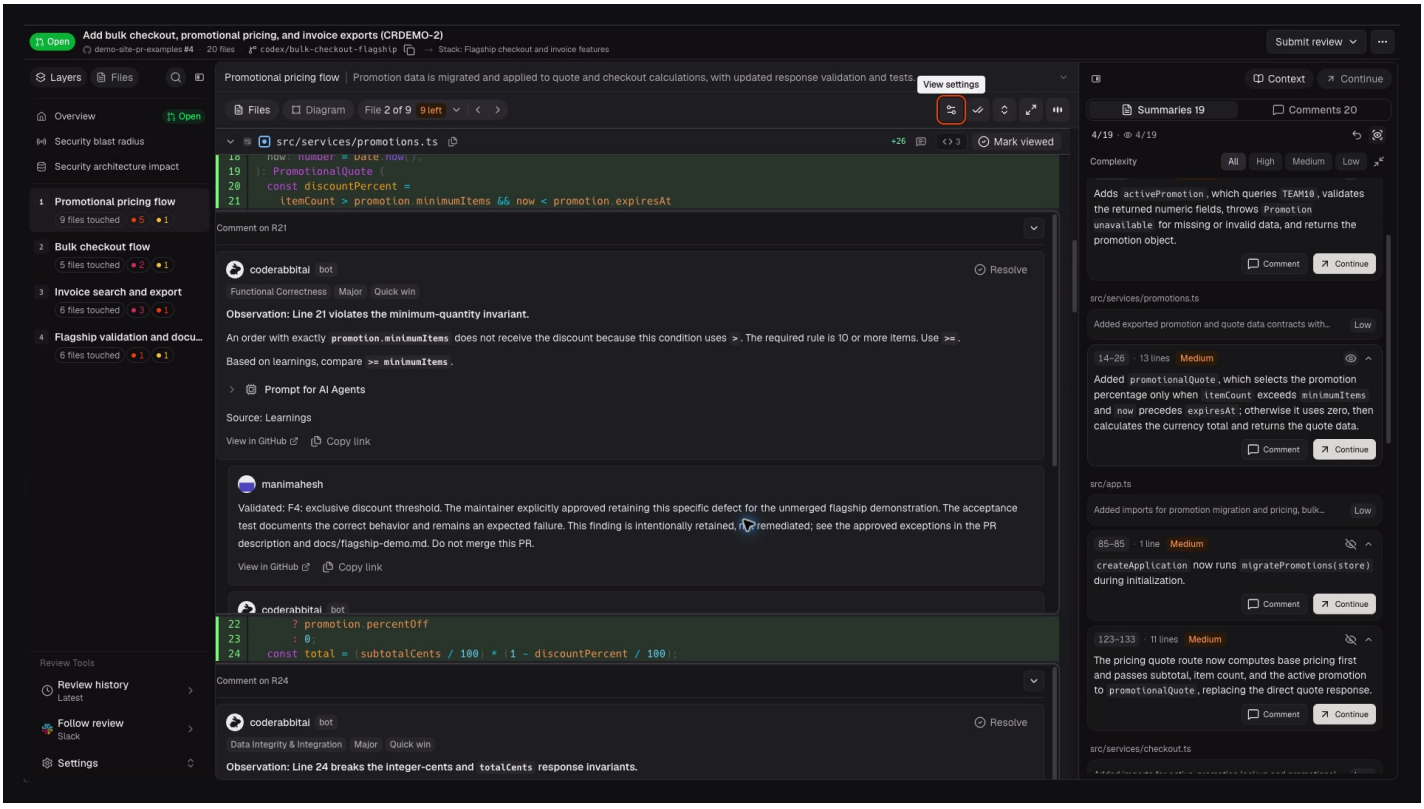
Scroll to the strict > comparison. Expand the historical discount thread; enable Resolved if it is hidden.

PR #4: historical discount thread

SAY

Next, I look at the discount comparison. The requirement includes an order of exactly 10 items while the strict, greater than condition excludes it. That discount discussion is retained in this demo's review history. I use it to examine the behavior without presenting it as a newly open finding.

SEE / VIDEO FRAME AT 01:31



WALK THE LAYERS / STEP 4 OF 4 / 01:43-01:51

Connect the behavior to tests

DO

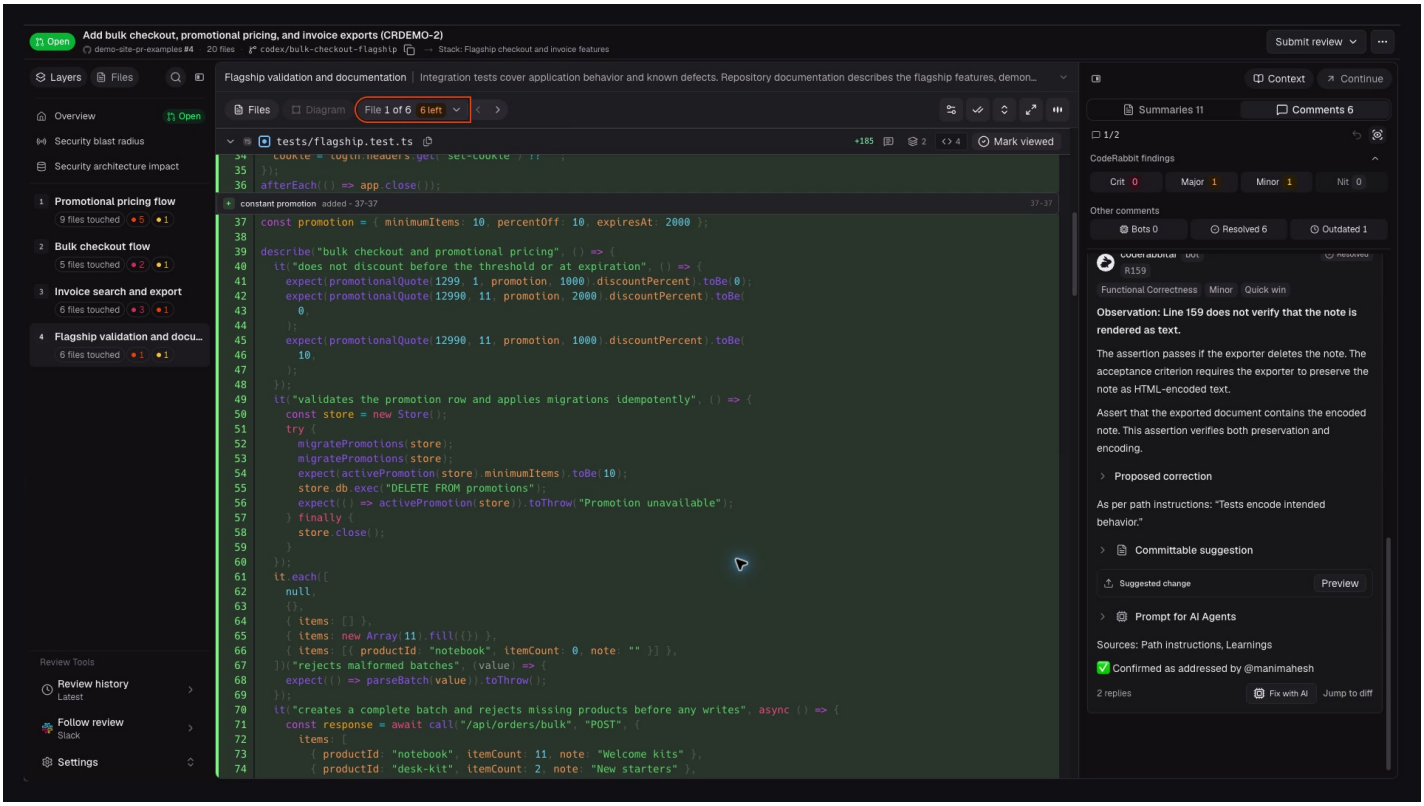
Open the tests/validation layer and select the relevant promotion test file.

Scroll to ordinary and exact-minimum cases. Keep code and findings beside the tests.

SAY

Finally, I look at the related tests. I want to see whether they check the behavior we expect at the boundary as well as an ordinary successful case.

SEE / VIDEO FRAME AT 01:50



FILTER FINDINGS / STEP 1 OF 4 / 01:52-01:58

Set the minimum severity

DO

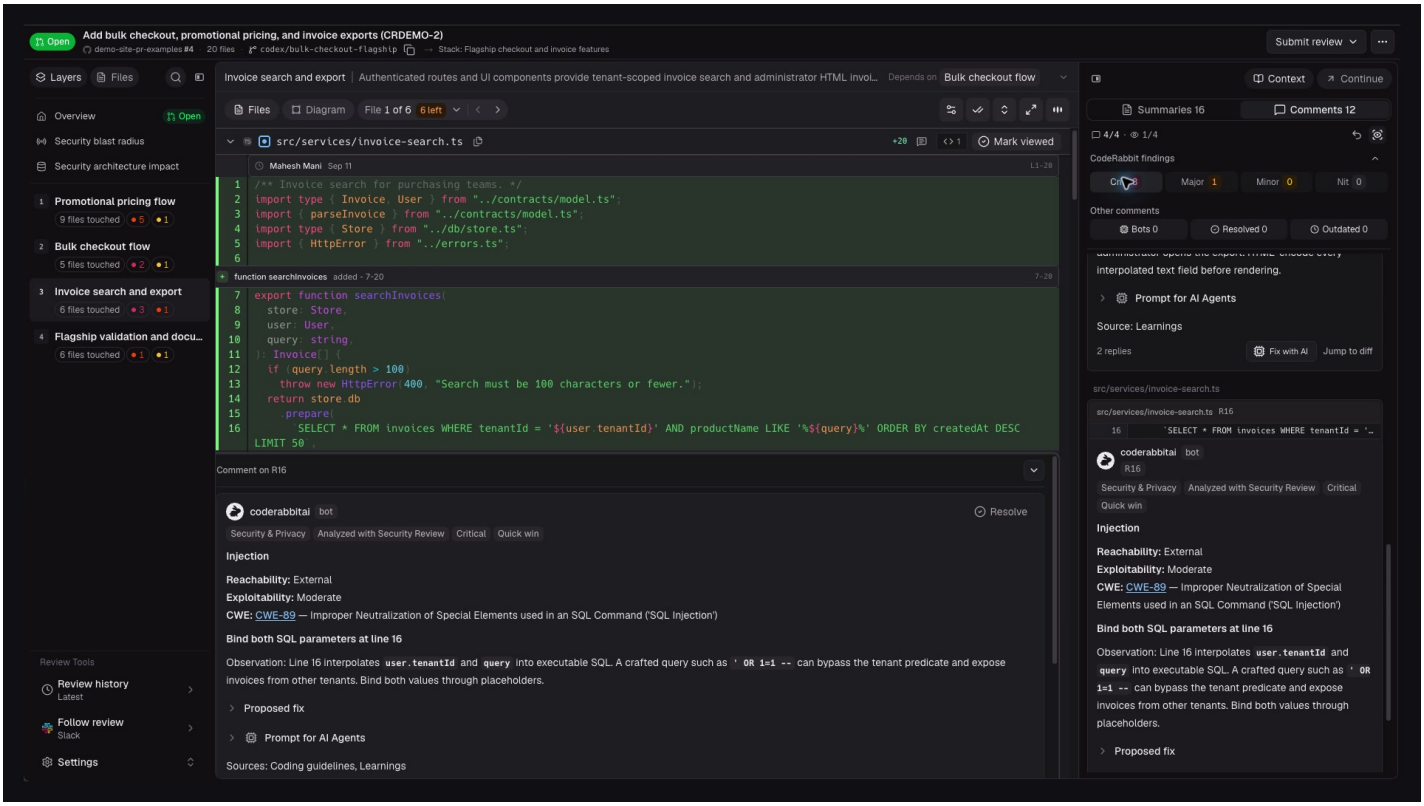
Open Comments in the review pane and click its filter controls.

Select the minimum severity for the security investigation.

SAY

I open the findings controls and set a minimum severity to focus on the more serious concerns.

SEE / VIDEO FRAME AT 01:57



FILTER FINDINGS / STEP 2 OF 4 / 01:59-02:20

Read the SQL finding beside the query

DO

Select the SQL injection finding in the filtered list.

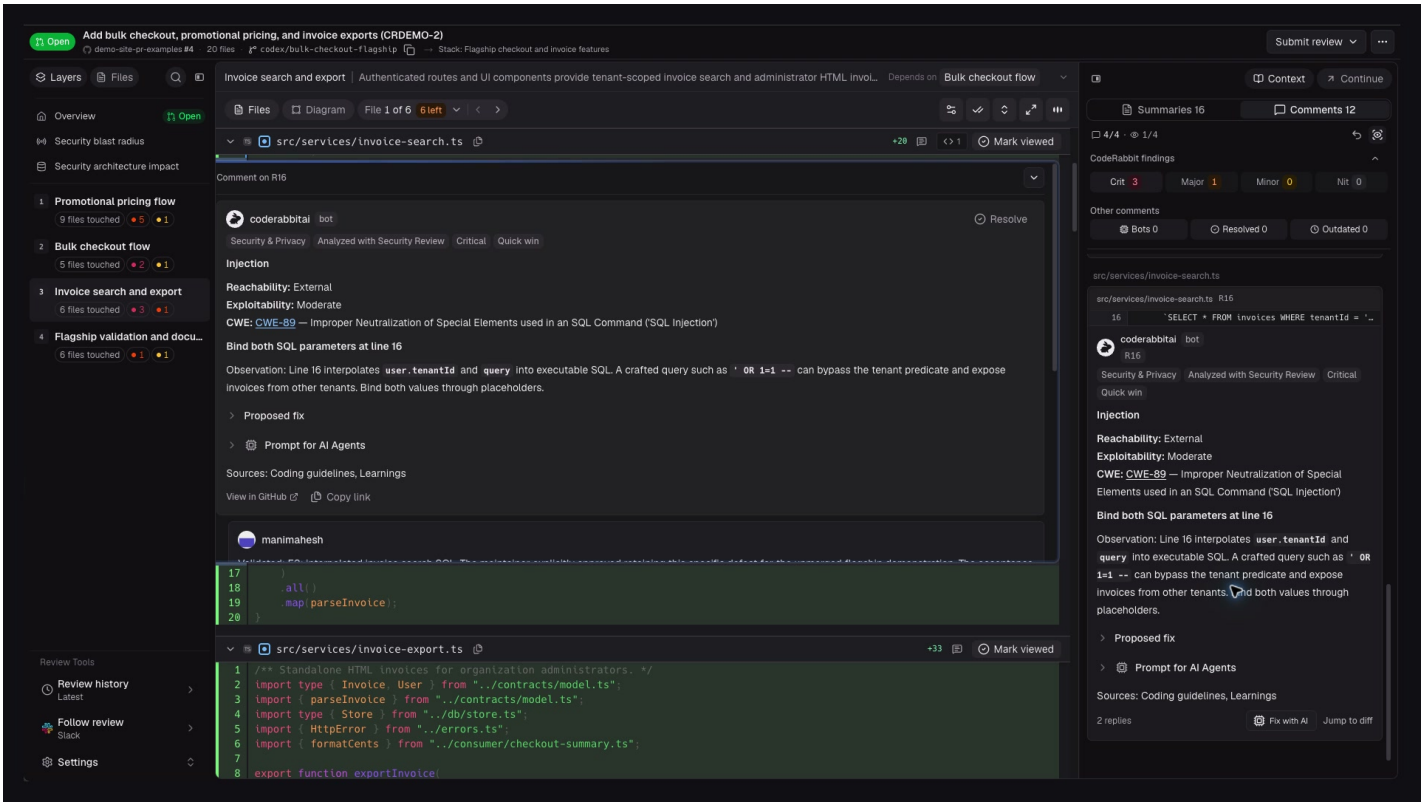
Scroll within its explanation to the tenant-boundary concern and compare it with the adjacent query.

PR #4: SQL finding

SAY

Then, I open the security finding beside the affected query. The explanation describes how search text can become part of a database command. I read it alongside the code and check what that means for the customer's data. A useful correction needs to keep the search within the customer's account, including when the input contains characters with special meaning in SQL.

SEE / VIDEO FRAME AT 02:08



FILTER FINDINGS / STEP 3 OF 4 / 02:21-02:33

Inspect resolved and outdated visibility

DO

Open the comment filters again. Show Resolved and Outdated controls.

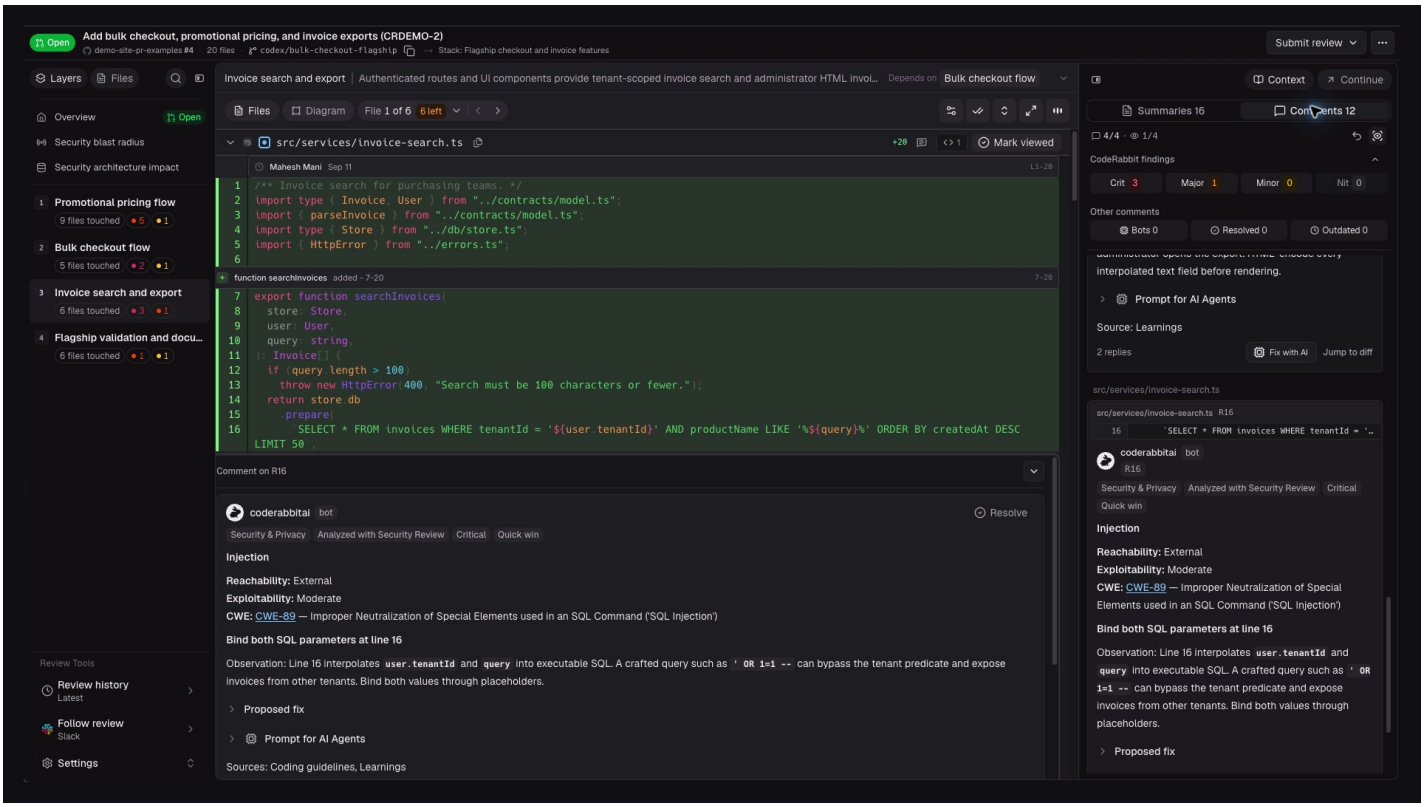
Enable Resolved to find the historical discount thread without treating it as a new unresolved finding.

PR #4: historical discount thread

SAY

I can also include resolved and outdated threads to revisit earlier discussions, such as the discount example. Their status matters when I am deciding what still needs work.

SEE / VIDEO FRAME AT 02:30



FILTER FINDINGS / STEP 4 OF 4 / 02:34-02:39

Check the filters before moving on

DO

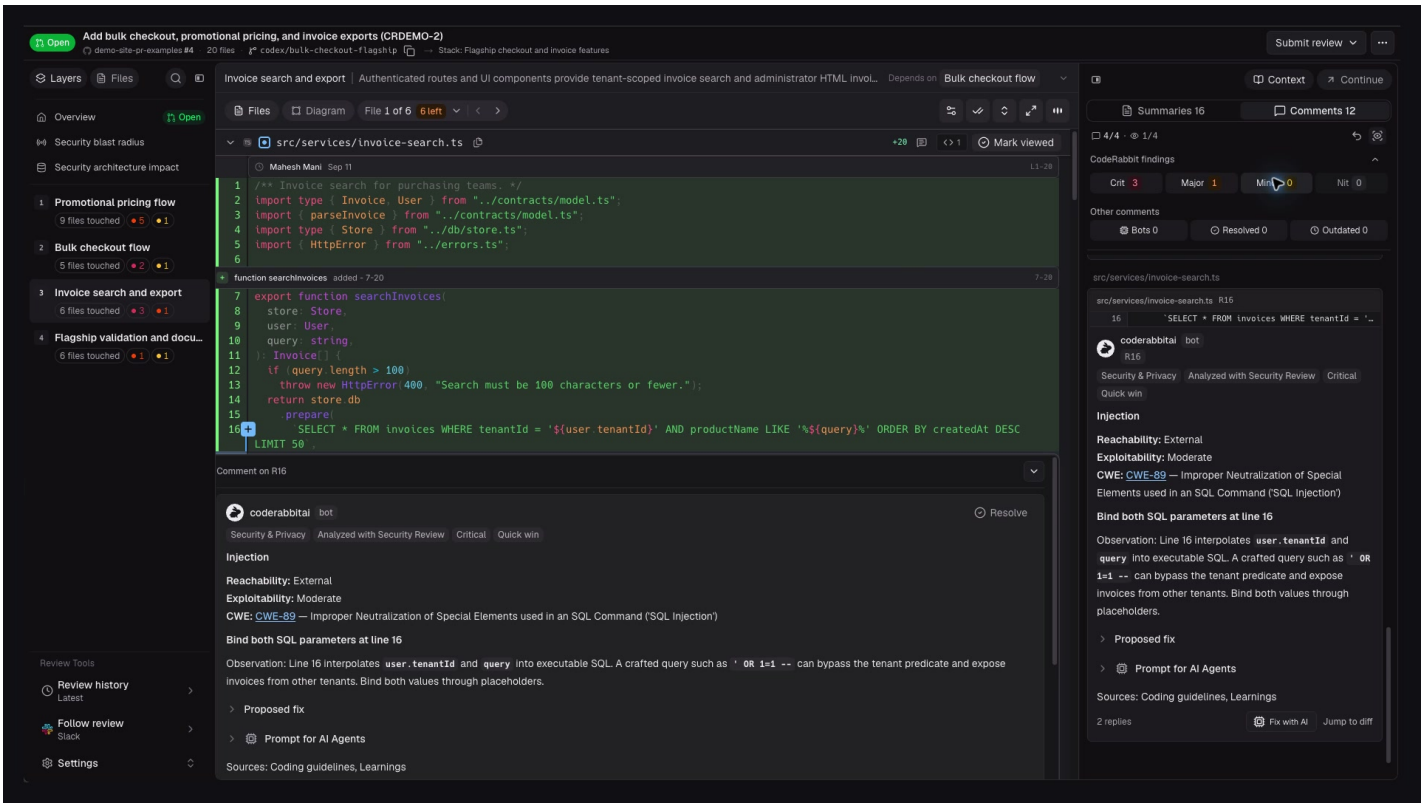
Return to the filters and check which findings are hidden.

Close the menu and keep the selected code visible.

SAY

Before finishing the review, I check the filters again so I don't overlook another finding I need to read.

SEE / VIDEO FRAME AT 02:38



ASK IN CONTEXT / STEP 1 OF 4 / 02:39-02:55

Open the inline conversation

DO

On promotions.ts, expand the discount item and select Continue.

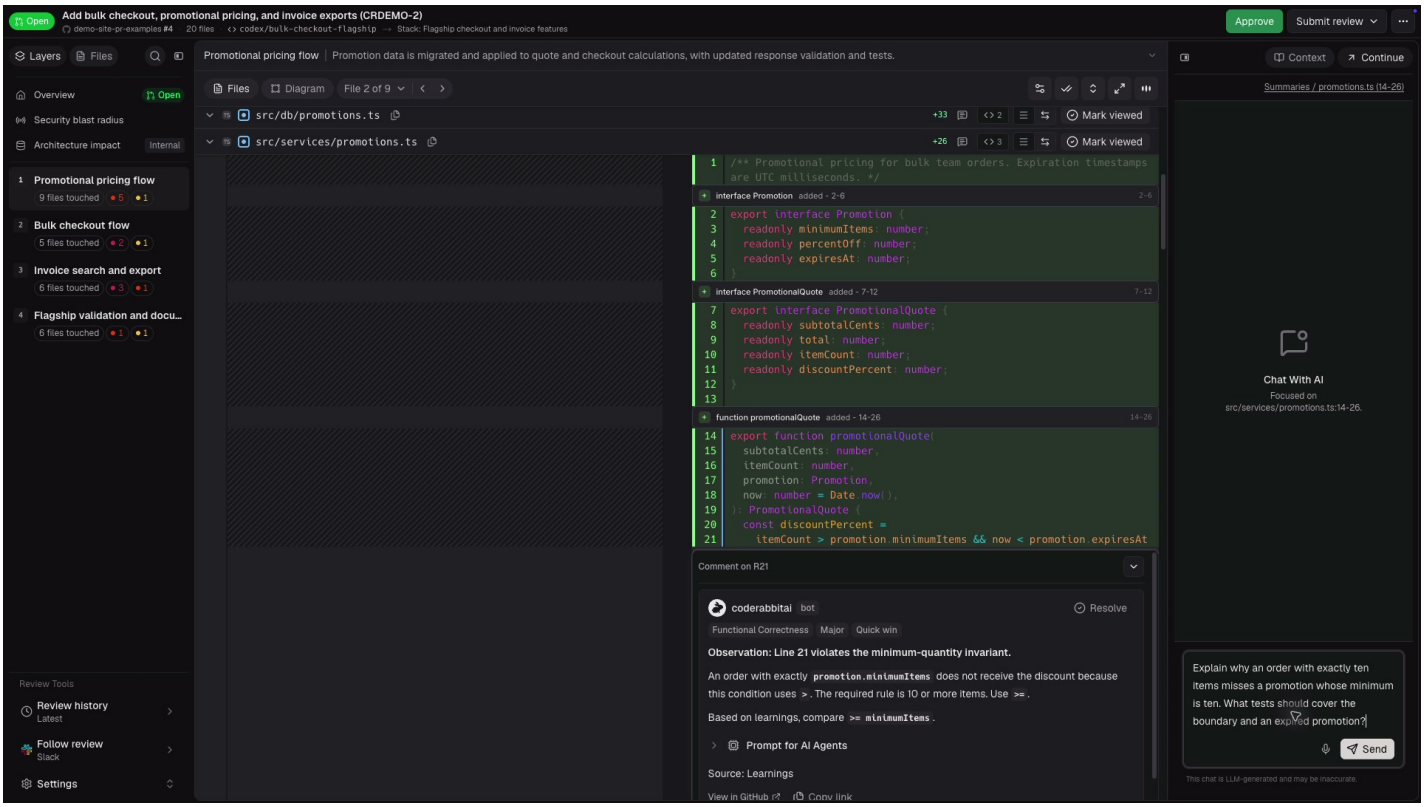
Click the conversation input. Ask what happens with ten items at a minimum of ten, including tests and expiration. Click Send for a training run.

PR #4: historical discount thread

SAY

On the discount's inline item, I select Continue to open a conversation about that code. I see what happens when an order contains 10 items and the minimum is 10. I also ask about tests and expirations because a quantity fix should leave an expired promotion ineligible.

SEE / VIDEO FRAME AT 02:53



ASK IN CONTEXT / STEP 2 OF 4 / 02:56-03:07

Read the clarified rule

DO

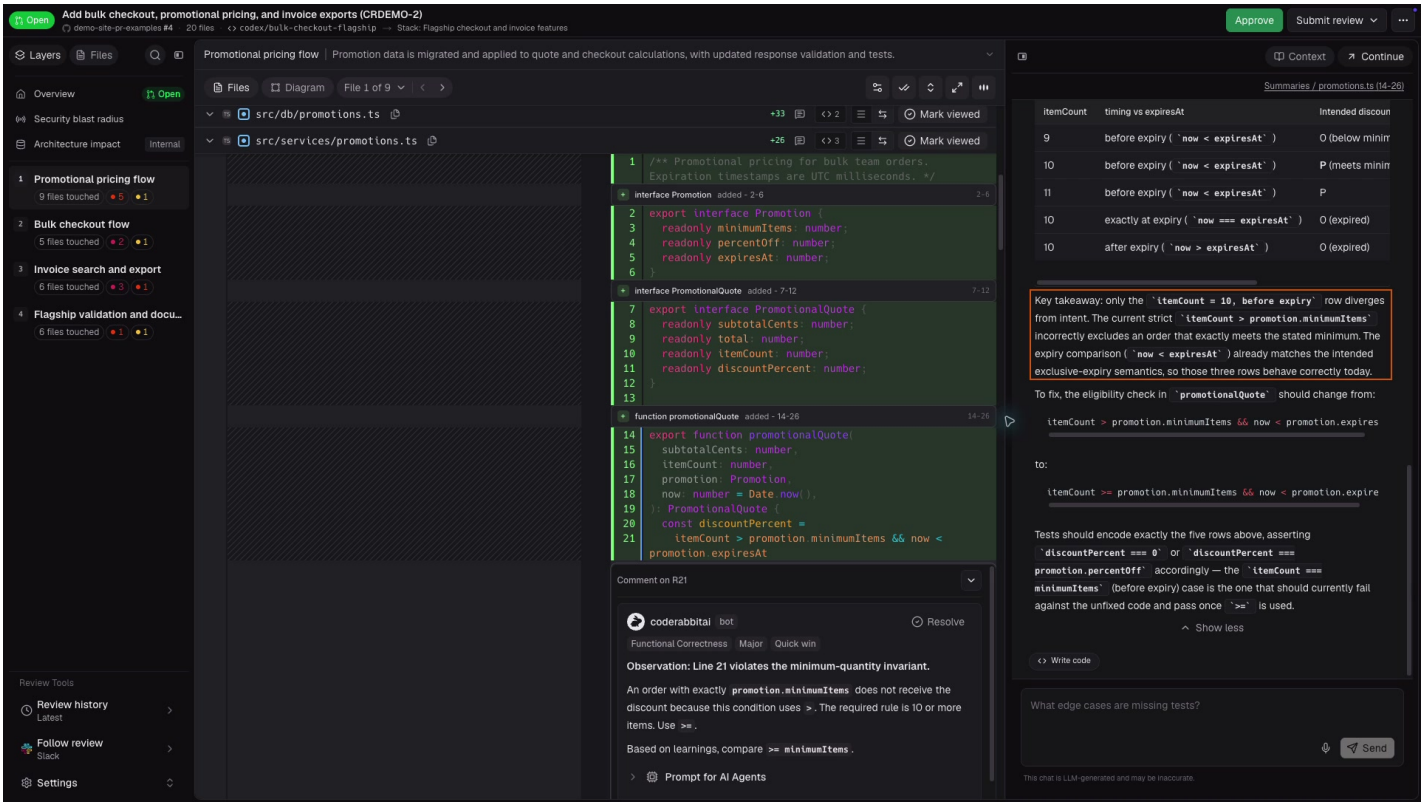
Scroll to the answer beside the selected code.

Keep the inclusive minimum and before-expiration explanation visible.

SAY

The first answer needs clarification. I restate the rule at least 10 items and only before the expiration time. Then, I compare the follow-up with the condition in the code.

SEE / VIDEO FRAME AT 03:04



ASK IN CONTEXT / STEP 3 OF 4 / 03:07-03:23

Request a compact boundary summary

DO

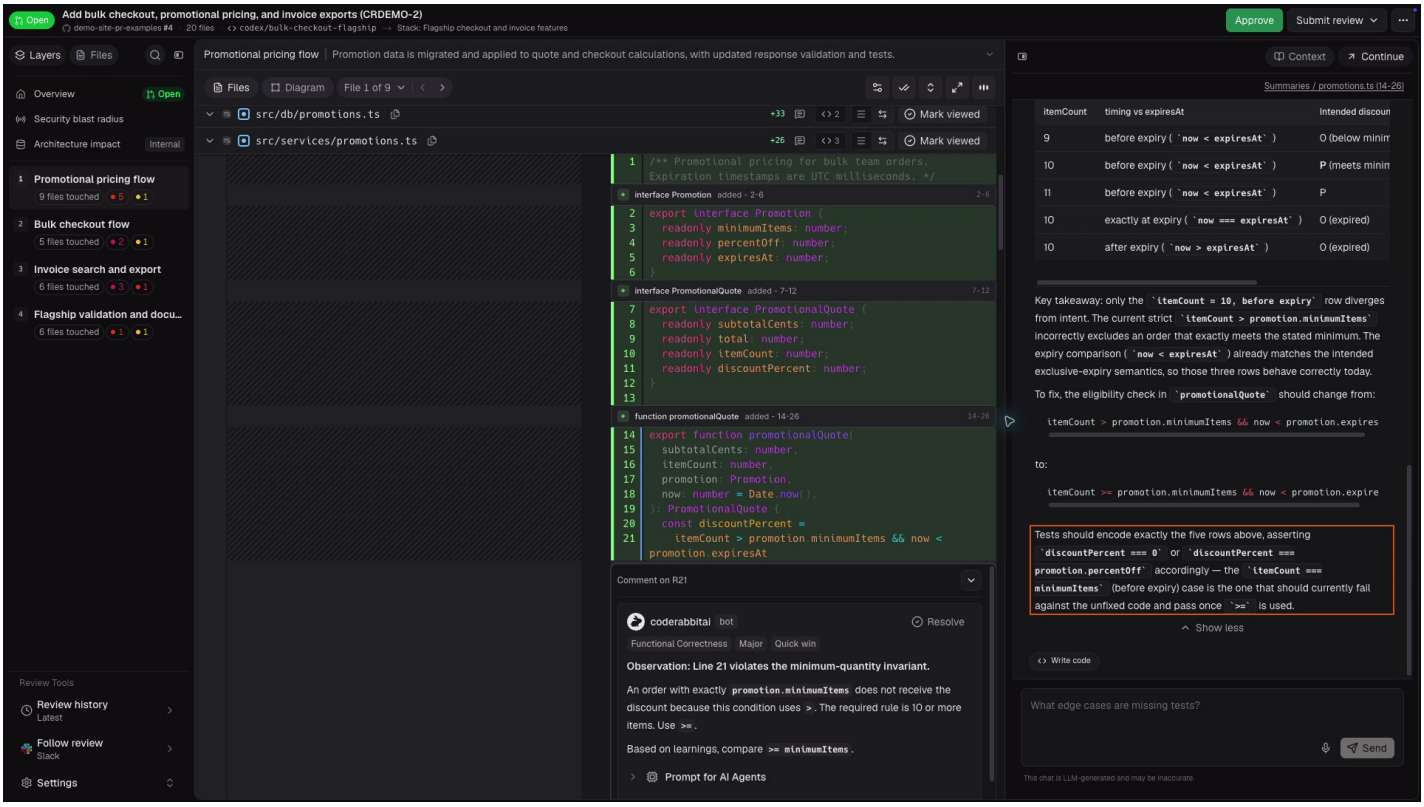
Scroll through the answer, then use the input to ask for the 9, 10 and 11 item cases and the expiration boundary.

Click Send for a training run; for rehearsal, inspect the existing answer. Keep the relevant code selected.

SAY

The checks we need are straightforward, below the minimum, exactly at it and above it. At the expiration time, the promotion should no longer apply. I ask for a short summary of those expectations. That gives me concrete behavior to verify in a proposed fix.

SEE / VIDEO FRAME AT 03:14



ASK IN CONTEXT / STEP 4 OF 4 / 03:23-03:30

Keep the conversation tied to its review

DO

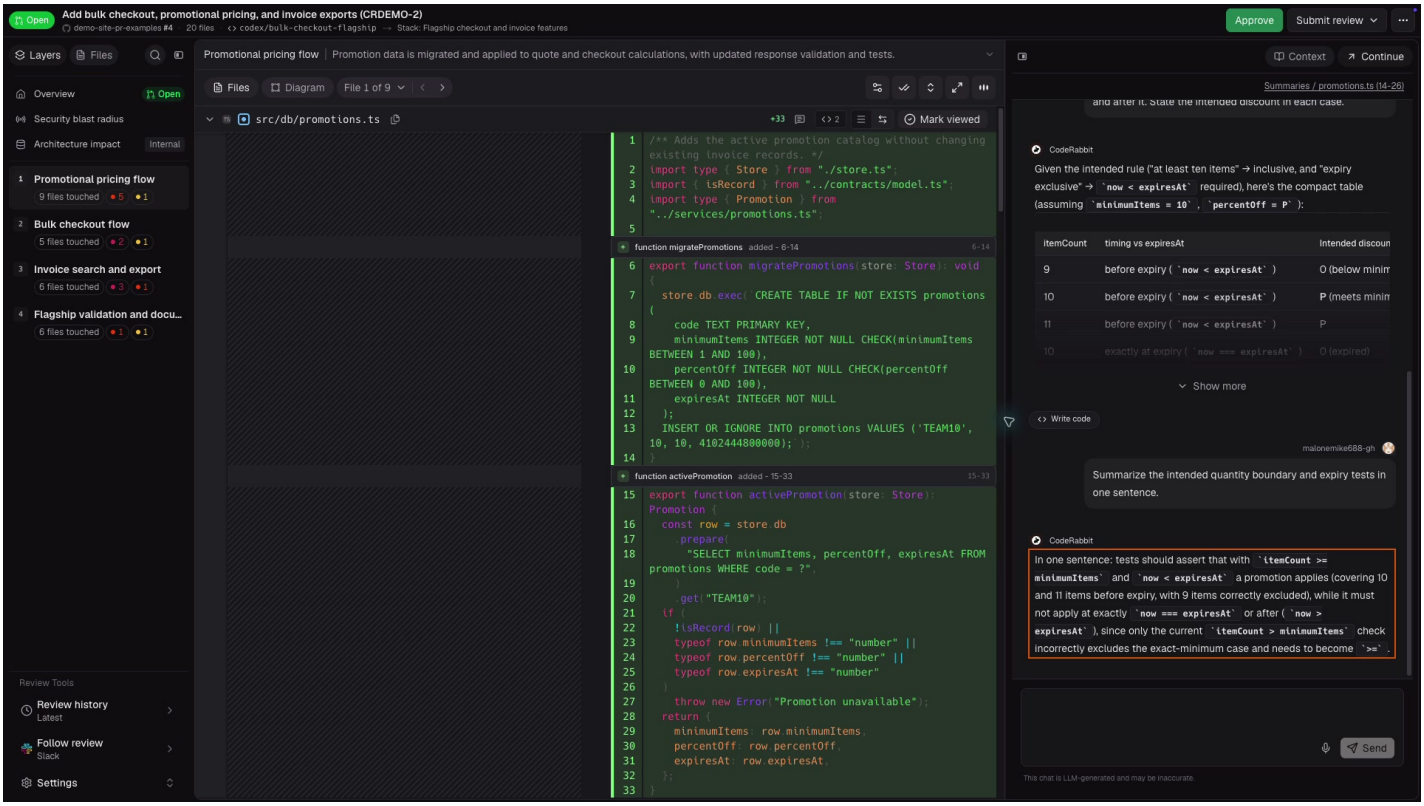
Read the response with the selected review version visible.

Use the same review for this conversation; open a new conversation if inspecting a later revision.

SAY

This conversation belongs to the review where it started. If the code changes, I'll open a new conversation on the updated review.

SEE / VIDEO FRAME AT 03:29



APPROVE & MERGE / STEP 1 OF 4 / 03:31-03:47

Preview the review choices

DO

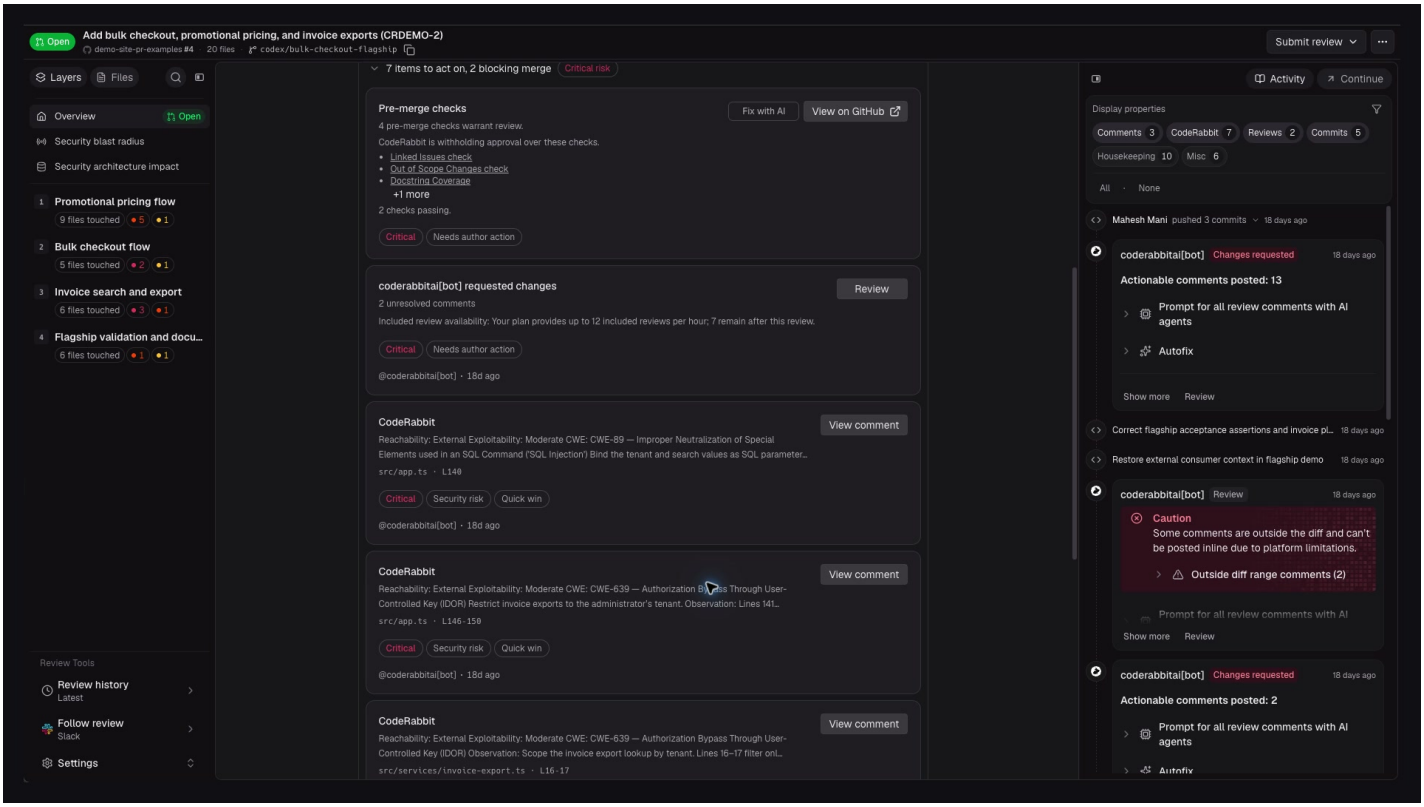
Open Submit review in Mike’s reviewer account.

Show Comment, Approve and Request changes. Close the dialog without submitting.

SAY

I open the review options. From this reviewer account, I can choose comment, approve or request changes. The invoice search concern still needs investigation. For this demo, I will leave the pull request open and preview the choices without submitting a review.

SEE / VIDEO FRAME AT 03:45



APPROVE & MERGE / STEP 2 OF 4 / 03:48-03:55

Return to the unresolved concerns

DO

Click Overview and scroll to remaining concerns.

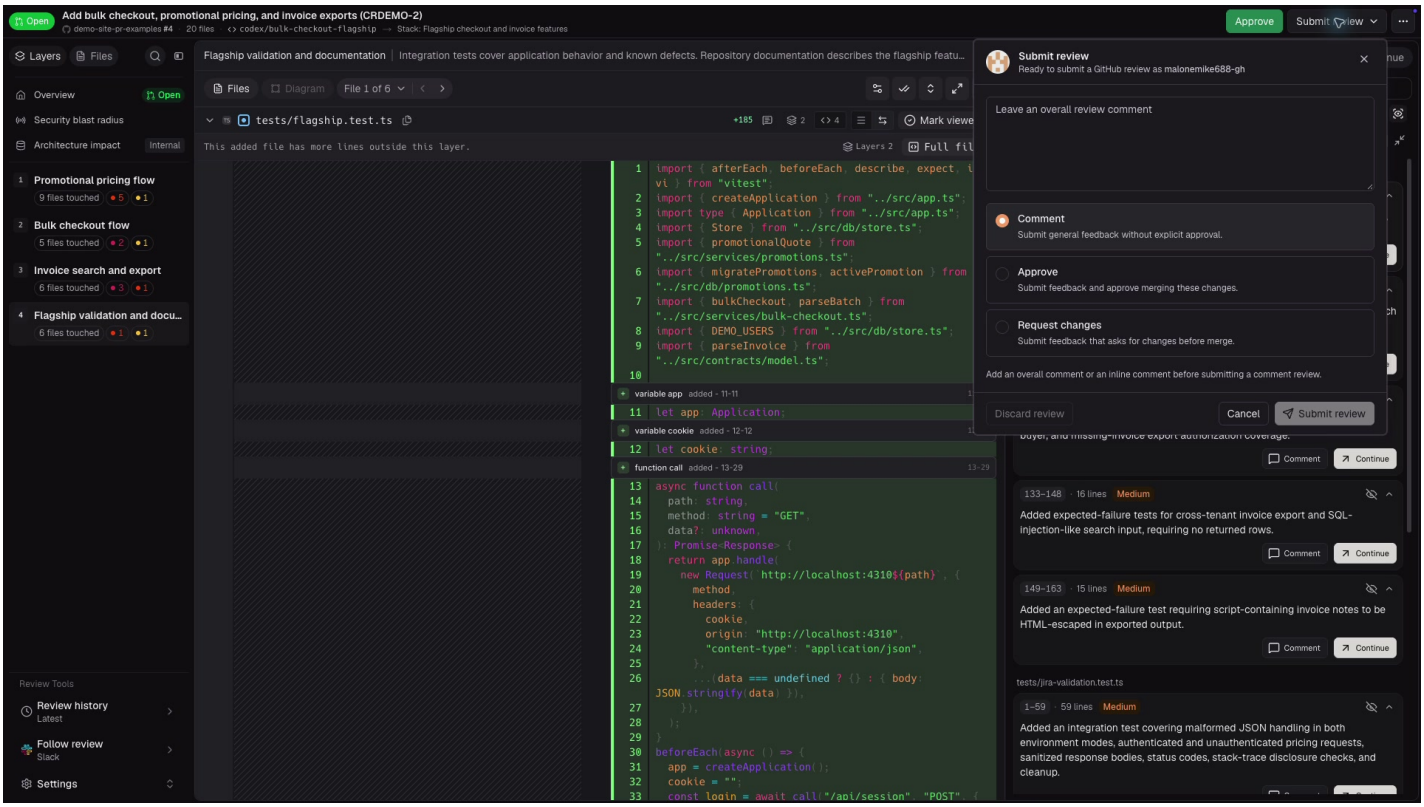
Keep the SQL finding and historical discount context visible; leave the fixture open.

PR #4: Change Stack

SAY

In a working review, I request changes needed to address the concern, then inspect the updated code and tests before approving.

SEE / VIDEO FRAME AT 03:54



APPROVE & MERGE / STEP 3 OF 4 / 03:56-04:14

Inspect provider merge requirements

DO

Open GitHub PR #4 and scroll to the merge area in Conversation.

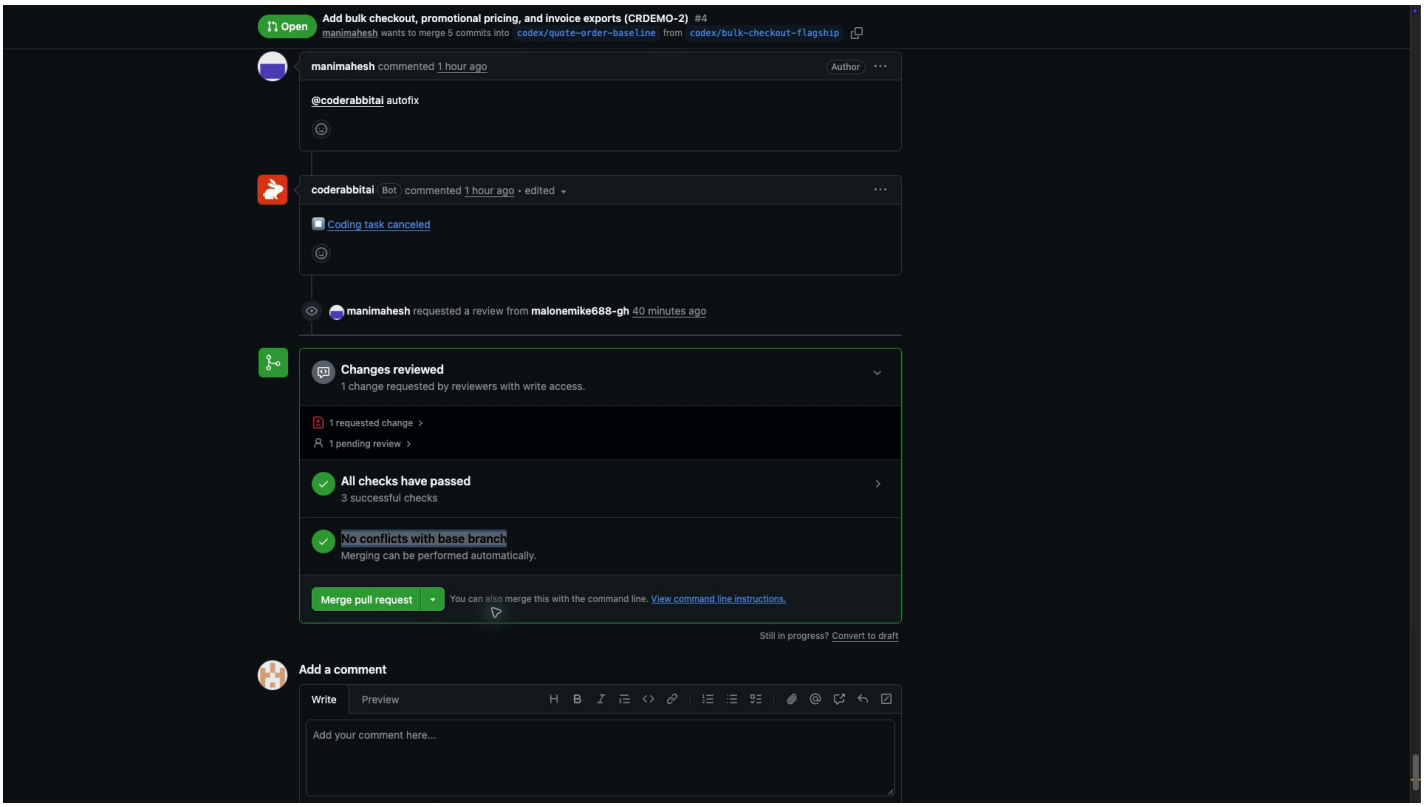
Expand the review and check details. Read available merge controls without activating them.

Flagship PR #4

SAY

I also check the repository's merge requirements. CodeRabbit's assessment informs the decision while the provider's branch rules determine which checks and approvals are enforced. A request for changes on its own doesn't prove that a merge is blocked. I need to confirm the rules that apply to this branch.

SEE / VIDEO FRAME AT 04:09



APPROVE & MERGE / STEP 4 OF 4 / 04:14-04:24

Finish on the review decision

DO

Return to Change Stack Overview and keep the remaining concerns visible.

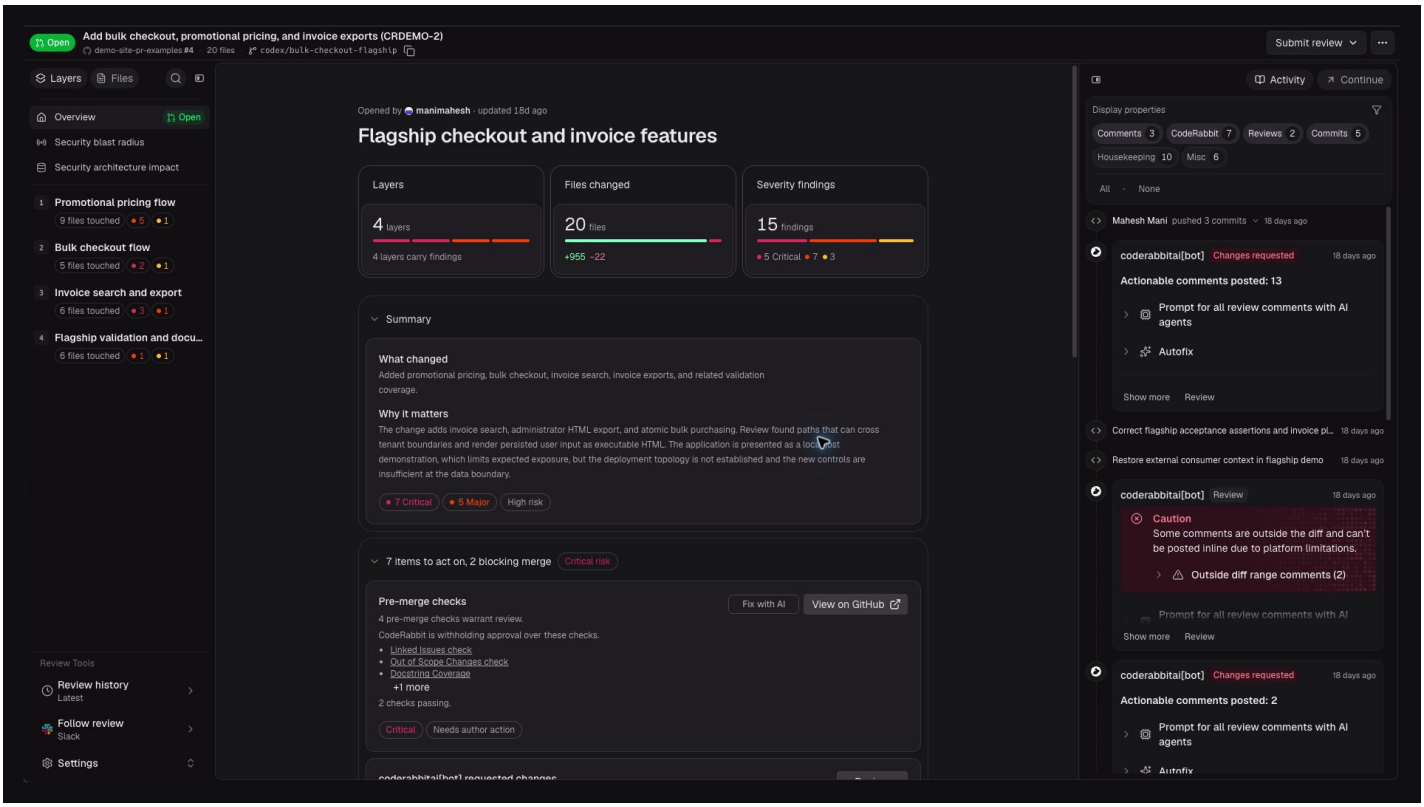
Finish the narration, then show the flow slide during the silent ending.

PR #4: Change Stack

SAY

We follow the change through its layers, check the findings and clarify the expected behavior. We can now explain what needs to happen before the team accepts it.

SEE / VIDEO FRAME AT 04:22



Resource directory

Links open the recorded examples. Historical comments and pinned source revisions preserve the relevant context when the live PR has moved on.

Flagship PR #4

https://github.com/coderabbit-demo/demo-site-pr-examples/pull/4

PR #4: Change Stack

https://app.coderabbit.ai/change-stack/coderabbit-demo/demo-site-pr-examples/pull/4/overview

PR #4: SQL finding

https://github.com/coderabbit-demo/demo-site-pr-examples/pull/4#discussion_r3991696811

PR #4: historical discount thread

https://github.com/coderabbit-demo/demo-site-pr-examples/pull/4#discussion_r3991696822

Slide deck

https://docs.google.com/presentation/d/1zk6UD0ZoAxtGRqDTQL-MVulAWu5MH4nYmd-9xJBpU5Y/edit

Triage queue

https://app.coderabbit.ai/triage